

High Availability in OpenStack

A practical architectural guide for cloud engineers and OpenStack administrators – covering the design patterns, components, and failure recovery strategies that keep your control plane running in production.

OPENSTACK HA ARCHITECTURE



What is High Availability?

Core Definition

High Availability (HA) is a design approach that ensures OpenStack services remain operational even when one or more infrastructure components fail – transparently, and without manual intervention.

- ❑ Without HA, a single controller failure halts all API operations – even though tenant VMs keep running.

HA Design Objectives

- **Eliminate SPOFs**
Remove every Single Point of Failure from the control plane path.
- **Automatic Failover**
Detect failures and reroute traffic without operator intervention.
- **Continuous Operations**
Maintain API availability, authentication, networking, and storage services at all times.

Single Controller vs. HA Architecture

✗ Single Controller — The Problem

All OpenStack services – Nova API, Keystone, Neutron, Glance, Cinder, RabbitMQ, MariaDB, and Horizon – run on one node. If it fails:

- Horizon becomes unavailable
- Authentication stops entirely
- VM provisioning halts
- Networking and storage operations freeze

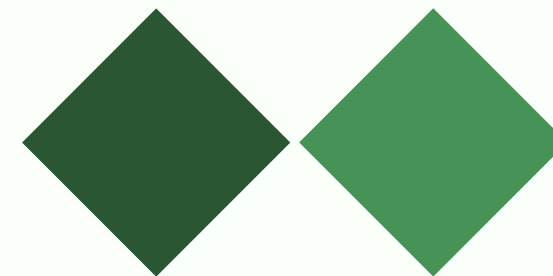
✓ HA Deployment — The Solution

Traffic enters through a Virtual IP managed by Keepalived, passes through HAProxy, and is distributed across three active controllers.

- No single point of failure
- Automatic failover via VRRP election
- Continuous API availability at all times

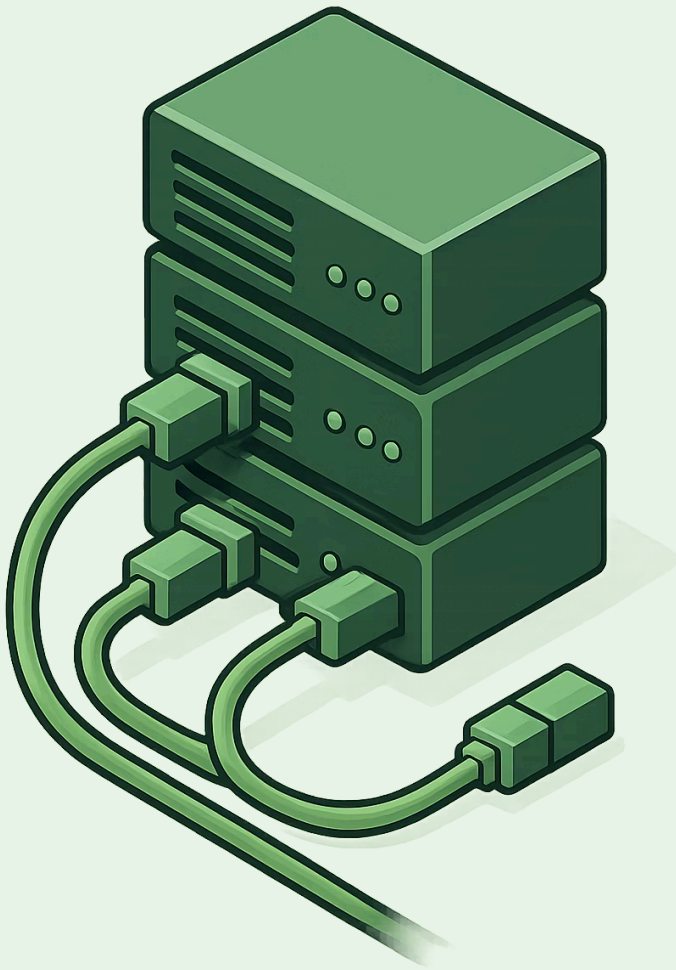
HA Cluster

VIP + HAProxy + 3 controllers; automatic failover.



Single Controller

All services on one node; full outage risk.



OpenStack HA Component Stack

Each layer of the HA stack solves a distinct failure domain. Together they eliminate SPOFs across networking, API, database, and messaging tiers.



Keepalived

Floating Virtual IP via VRRP. Provides a single, stable endpoint for all users regardless of which controller is active master.



HAProxy

Layer-4/7 load balancer. Distributes API requests across healthy controllers and automatically removes failed nodes from rotation.



Galera

Synchronous multi-master MariaDB cluster. Every controller holds an identical, always-current copy of all OpenStack databases.



RabbitMQ

Clustered message broker. Ensures asynchronous inter-service communication survives individual node failures.



Memcached

Distributed token and session cache. Reduces Keystone database load and improves authentication performance at scale.



Controllers (×3)

Each node runs the full API service stack: Keystone, Nova API, Neutron, Glance, Cinder, and Horizon.

Why Exactly Three Controllers?

The magic number in distributed systems is three – it's the minimum node count that supports majority quorum voting and avoids the split-brain problem.

1

1 Controller

Any failure takes the entire cloud control plane offline. No redundancy whatsoever.

2

2 Controllers

Introduces the **split-brain problem**: if the two nodes lose contact, each believes it is the authoritative master. Neither can safely proceed, causing a stalemate.

3

3 Controllers

Majority quorum becomes possible. Two surviving nodes can always outvote one, allowing the cluster to elect a new master and continue operations automatically. This is the recommended production architecture.

- ❏ Three controllers tolerate one full node failure while maintaining quorum. For higher fault tolerance, use five controllers (tolerates two simultaneous failures).

Galera: Synchronous Multi-Master Database Clustering

Traditional MySQL Replication — Limitations

- Asynchronous replication creates lag between master and replicas
- A crash before replication completes causes data loss
- Only one node accepts writes – a SPOF by design

Galera — How It Works

Every write operation is broadcast to all nodes before the transaction commits. All nodes acknowledge the write simultaneously. If any node rejects it, the transaction is rolled back on all nodes.

Galera Advantages for OpenStack

Multi-Master

Any controller can accept database writes – no write bottleneck.

Synchronous

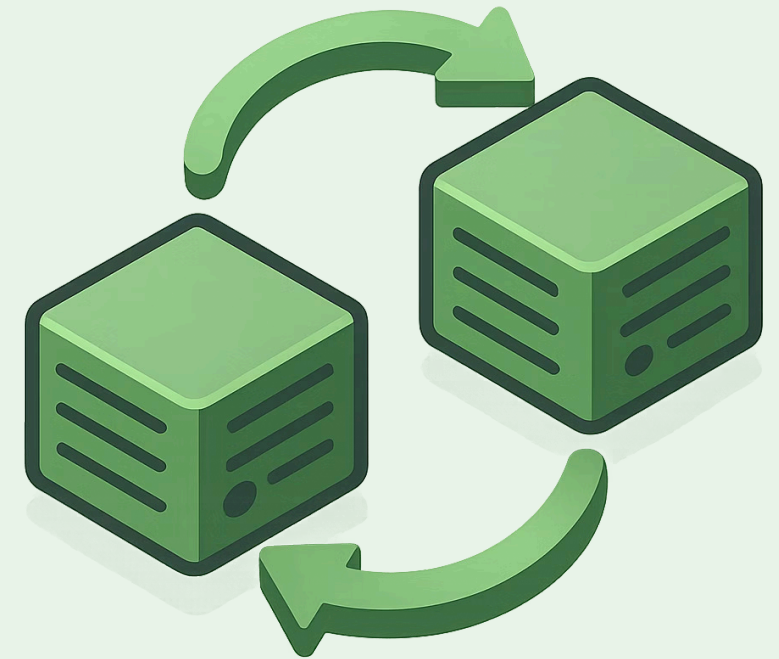
Zero replication lag. All nodes are always consistent.

Auto Recovery

A rejoining node automatically catches up via State Snapshot Transfer (SST).

No Manual Failover

Quorum is maintained automatically when a node fails.



What Galera Protects: OpenStack Database Scope

Galera replicates the databases for every major OpenStack service. Any controller in the cluster can serve reads and writes for all of them.

Keystone

- Users & credentials
- Roles & assignments
- Projects & domains

Nova

- Instances & flavors
- Host aggregates
- Live migration records

Neutron

- Networks & subnets
- Routers & ports
- Security groups & floating IPs

Cinder

- Volumes & attachments
- Snapshots & backups

Glance

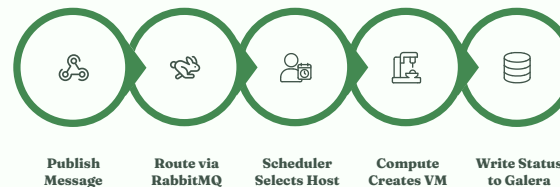
- Image metadata & locations
- Visibility & ownership

RabbitMQ: The Messaging Backbone

Why Asynchronous Messaging?

OpenStack services never call each other directly. Instead, they communicate via RabbitMQ message queues. This decouples services, improves resilience, and allows each component to scale independently.

A Typical VM Creation Flow



What Breaks Without RabbitMQ HA?

Nova Scheduling Stops

New VMs cannot be placed on compute hosts.

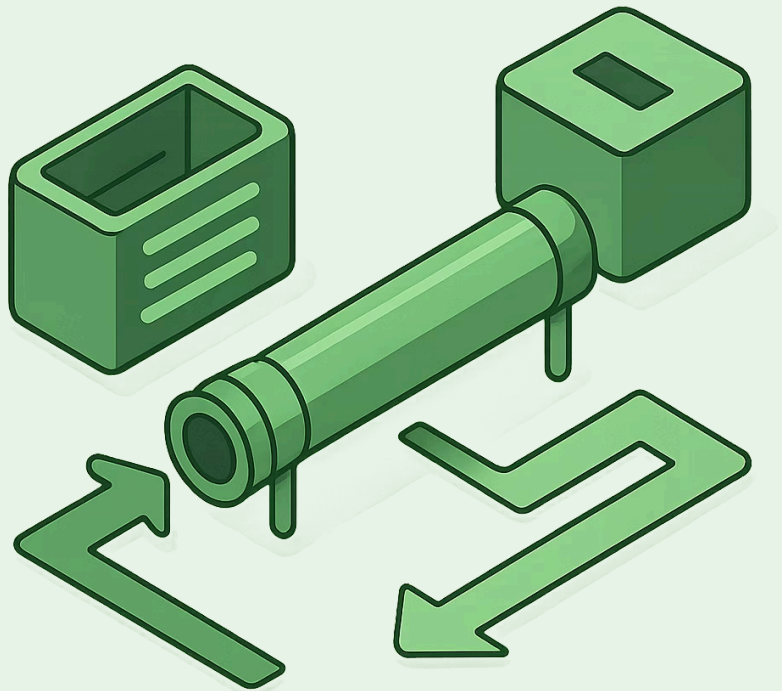
Neutron Stalls

Network provisioning requests queue up and time out.

Cinder Blocks

Volume attach and detach operations cannot complete.

A clustered RabbitMQ (three nodes, mirrored queues) ensures messages remain available even when one broker node fails.



HAProxy: Load Balancing & Health Checking

HAProxy sits behind the Virtual IP and is the first component that actually processes user API traffic. It serves two critical functions: distributing load and detecting failures.

Load Distribution

Incoming requests are round-robin across healthy controller nodes. No single controller becomes a bottleneck under high API load.

- Request 1 → Controller1
- Request 2 → Controller2
- Request 3 → Controller3

Continuous Health Checks

HAProxy actively probes each backend API endpoint on a configurable interval (typically every 2 - 5 seconds). If a controller fails its health check:

- It is immediately removed from the active pool
- Existing connections are drained gracefully
- Traffic redistributes to remaining healthy controllers
- No operator action required – no DNS changes needed

Keepalived: Virtual IP Failover via VRRP

Keepalived implements the **Virtual Router Redundancy Protocol (VRRP)** to provide a single, stable IP address – the VIP – that floats between controllers based on health and priority.

1 Normal State

Controller1 holds MASTER status. VIP 10.0.0.100 is bound to Controller1's interface. All users connect to the VIP.

2 Failure Detected

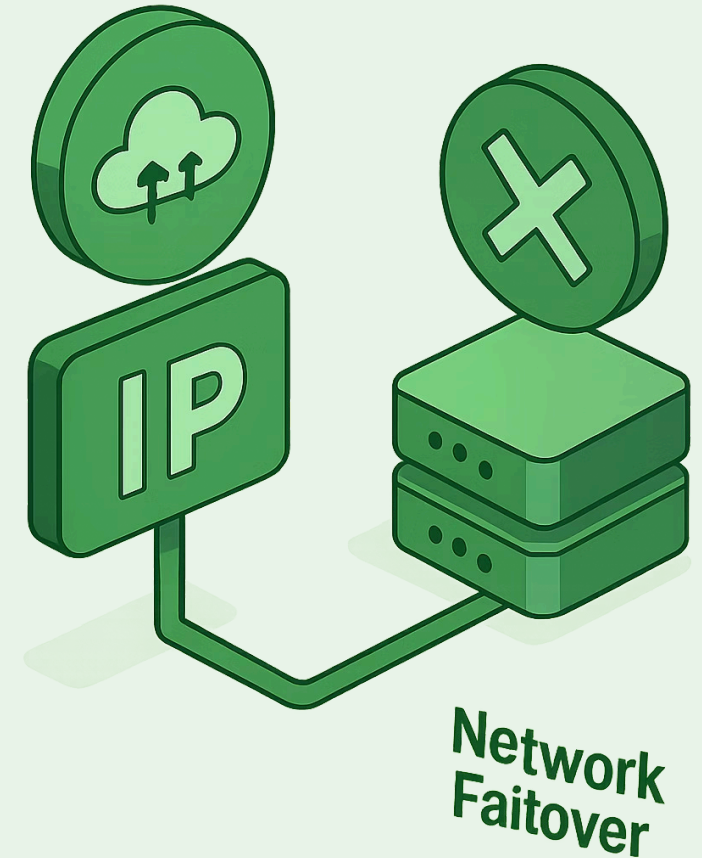
Controller1 stops sending VRRP advertisements. Controllers 2 and 3 detect the silence after a configurable dead interval (default: ~3 seconds).

3 VRRP Election

The remaining controllers hold a priority-based election. The node with the highest priority value wins MASTER status.

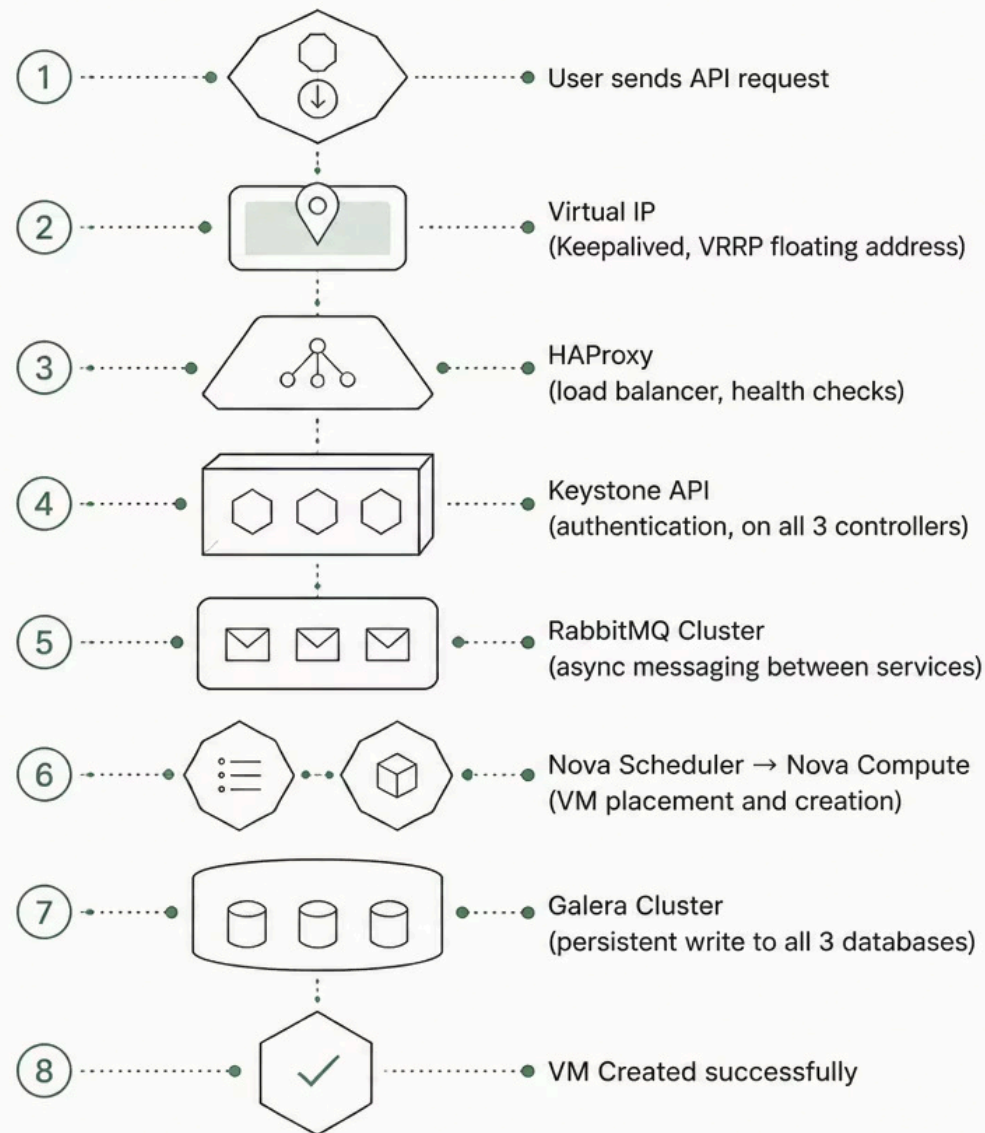
4 VIP Migrates

Controller2 assumes the VIP. Users at 10.0.0.100 continue operating – no DNS change, no reconfiguration required.



Complete HA Request Flow

Every user API call traverses the full HA stack. Each layer independently tolerates failure, so the control plane survives individual component outages.



Notice that every component in the chain – VIP, proxy, API, messaging, and database – has its own clustering or redundancy mechanism. The result is a control plane with no single component whose failure stops cloud operations.

Failure Scenario: Controller2 Goes Down

Let's trace exactly what happens – automatically, within seconds – when Controller2 fails unexpectedly in a production HA cluster.



Keepalived

If Controller2 held MASTER, VRRP advertisement stops. Remaining nodes elect a new MASTER within ~3 seconds. VIP migrates automatically.



HAProxy

Health check to Controller2 fails. It is removed from all backend pools immediately. Requests 1 and 3 continue on Controllers 1 and 3.



Galera

Two remaining nodes still form a quorum majority. Database reads and writes proceed normally. No data is lost.



RabbitMQ

Mirrored queues on Controllers 1 and 3 contain all messages. Queue consumers reconnect automatically. No messages are lost.

✅ Result: Existing VMs keep running. Horizon stays accessible. APIs remain available. New VMs can still be provisioned – all without operator intervention.

Component Responsibilities at a Glance

A concise reference mapping each HA component to its role and the specific failure it protects against.

Component	Role	Why It's Needed
Keepalived	Virtual IP failover (VRRP)	Provides one stable, DNS-independent endpoint; migrates automatically on controller failure
HAProxy	API load balancing & health checks	Distributes traffic across controllers and removes failed nodes from rotation without operator action
Galera	Synchronous MariaDB clustering	Keeps all OpenStack service databases consistent and writable across all three controllers
RabbitMQ	Clustered message broker	Ensures inter-service messaging survives individual broker failures; no message loss with mirrored queues
Memcached	Token & session cache	Reduces Keystone database pressure; improves authentication throughput at scale
3× Controllers	Full API service stack on each node	Provide quorum for Galera and RabbitMQ; tolerate one full node failure without service interruption



Key Takeaways

Three Controllers Are the Minimum

One tolerates the split-brain problem; three solve it. Quorum-based majority voting enables automatic cluster recovery.

Every Layer Is Independently Redundant

VIP (Keepalived), proxy (HAProxy), API (3× controllers), messaging (RabbitMQ cluster), and database (Galera) each survive individual failures – defense in depth.

Tenant VMs Are Not Affected by Control Plane Failures

Running instances continue operating on compute nodes. HA protects the ability to *manage* the cloud – provisioning, networking, and storage operations – not the data plane itself.

Automation Is the Goal

A well-designed HA cluster detects, isolates, and recovers from failures in seconds – without pager alerts, DNS changes, or manual failover commands.